Lista de Exercícios - Curso de Estruturas de Dados I (em C/C++)
Prof. Igor Machado Coelho - Tópico: Árvores (Updated 10/06/2025)

Observação: os exercícios devem ser feitos em C/C++ (ou similar!). Foque mais na lógica do que em erros básicos de programação e SEMPRE discuta a proposta do algoritmo (não quero apenas código!). **Sempre** analise a complexidade assintótica dos métodos implementados. É permitido o uso da STL em C++, mas não é permitido uso da sobrecarga do **método *operator=*** para *referências lvalue* ou *rvalue* (ou seja, somente utilize **=** com tipos primitivos como int, float, char*, etc, nunca agregados ou classes). Não é permitido o uso de métodos de *#include<algorithm>*.

Parte I: Estruturas em Árvores Binárias

1. Considere a seguinte implementação de árvore binária:

```
struct NoBin {
    char chave;
    NoBin* esq;
    NoBin* dir;
};
```

1.a) Implemente um algoritmo que detecta se uma árvore é estritamente binária: auto estritaBin(NoBin* no) -> bool;
- Obs: árvore vazia é estritamente binária

1.b) Implemente um algoritmo que imprime a árvore em níveis: auto emNiveis(NoBin* no) -> void;
1.c) Implemente um algoritmo que detecta se uma árvore é perfeita: auto ehPerfeita(NoBin* no) -> bool;
- Obs: árvore vazia é perfeita

1.d) Implemente um algoritmo que detecta se uma árvore é completa: auto ehCompleta(NoBin* no) -> bool;
- Obs: árvore vazia é completa

1.e) Implemente um algoritmo que calcula a altura de um nó: auto altura(NoBin* no) -> int;
1.f) Implemente um algoritmo que imprime a árvore no percurso de preordem: auto preordem(NoBin* no) -> void;
1.g) Implemente um algoritmo que imprime a árvore no percurso de posordem: auto posordem(NoBin* no) -> void;
1.h) Implemente um algoritmo que imprime a árvore no percurso de inorder: auto inorder(NoBin* no) -> void;

Parte II: Estruturas em Árvores Binárias de Busca

2. Implemente um algoritmo que insere um novo elemento em uma árvore binária de busca: auto insere(char chave, NoBin* no) -> void;
3. Implemente um algoritmo que encontra o maior valor na árvore: auto maior(NoBin* no) -> char;
- Obs: considere somente letras maiúsculas, de B..X, e use o caractere ‘A’ como menor valor possível

4. Implemente um algoritmo que encontra o menor valor na árvore: auto menor(NoBin* no) -> char;
- Obs: considere somente letras maiúsculas, de B..X, e use o caractere ‘Z’ como maior valor possível

5. Implemente um algoritmo que verifica se uma árvore é binária de busca: auto ehBinariaBusca(NoBin* no) -> bool;
6. Implemente um algoritmo que encontra um elemento em uma árvore binária de busca: auto busca(char chave, NoBin* no) -> char;
- Obs: retorne o caractere ‘\0’ caso nenhum seja encontrado

7. Implemente um algoritmo que encontra o elemento imediatamente anterior a uma dada chave em uma árvore binária de busca: auto anterior(char chave, NoBin* no) -> char;

8. Implemente um algoritmo que encontra o elemento imediatamente posterior a uma dada chave em uma árvore binária de busca: auto posterior(char chave, NoBin* no) -> char;

Parte III: Estruturas em Árvores AVL

9. Implemente um algoritmo que calcula o fator de balanceamento de um no: auto fator(NoBin* no) -> int;
10. Implemente um algoritmo que imprime na tela qual dos 4 tipos de rotação é necessário em um dado nó, para uma AVL desbalanceada: auto qualRotacao(NoBin* no) -> void;

Parte IV: Estruturas em Filas de Prioridade

11. Implemente um algoritmo que verifica se uma arvore é um heap: auto ehHeap(int[] heap, int N) -> bool;
12. Implemente um algoritmo que adiciona um novo elemento em uma fila de prioridade: auto novo(int elemento, int[] heap, int N) -> void;